

МНОГОПОТОЧНОЕ ПРОГРАММИРОВАНИЕ

МНОГОПОТОЧНОЕ ПРОГРАММИРОВАНИЕ.....	1
Модуль 1: Синхронность vs Асинхронность.....	2
1.1 Чек-лист знаний.....	2
1.2 Глоссарий.....	2
1.3 Полезные ссылки.....	2
1.4 Задача на проверку.....	2
Модуль 2: Потоки (Threading) и Конкурентность.....	4
2.1 Чек-лист знаний.....	4
2.2 Глоссарий.....	4
2.3 Полезные ссылки.....	4
2.4 Задача на проверку.....	4
Модуль 3: Многопроцессность (Multiprocessing) и Параллелизм.....	6
3.1 Чек-лист знаний.....	6
3.2 Справочная таблица различий.....	6
3.3 Полезные ссылки.....	7
3.4 Задача на проверку.....	7
СБОРНИК ПРОВЕРОЧНЫХ ТЕСТОВ.....	8
Тест 1: Синхронность vs Асинхронность.....	8
Часть А: Выберите один правильный ответ.....	8
Часть Б: Задания со свободным ответом (напишите код).....	9
Тест 2: Потоки (Threading).....	10
Часть А: Выберите один правильный ответ.....	10
Часть Б: Задания со свободным ответом (напишите код).....	11
Тест 3: Многопроцессность (Multiprocessing).....	12
Часть А: Выберите один правильный ответ.....	12
Часть Б: Задания со свободным ответом (напишите код).....	13
ОТВЕТЫ И РАЗБОР ЗАДАНИЙ.....	14
Тест 1: Синхронность vs Асинхронность (asyncio).....	14
Тест 2: Потоки (Threading).....	15
Тест 3: Многопроцессность (Multiprocessing).....	16

Модуль 1: Синхронность vs Асинхронность

Что нужно уметь: Проектировать конкурентный код для I/O-bound задач, правильно использовать Event Loop, объявлять корутины и управлять ими.

1.1 Чек-лист знаний

- [] Понимание разницы между синхронным (блокирующим) и асинхронным (неблокирующим) выполнением.
- [] Объявление корутин с помощью `async def` и приостановка выполнения через `await`.
- [] Роль цикла событий (Event Loop) и запуск точки входа через `asyncio.run()`.
- [] Конкурентный запуск пула задач с помощью `asyncio.gather()`.
- [] Понимание природы I/O-bound задач (сетевые запросы, чтение диска, работа с БД).

1.2 Глоссарий

- **Корутина (Coroutine)** – специальная функция, выполнение которой может быть приостановлено и возобновлено. При вызове возвращает объект корутины, а не результат.
- **Цикл событий (Event Loop)** – ядро `asyncio`, которое управляет выполнением асинхронных задач, распределяет ресурсы и переключается между задачами, когда они ожидают завершения I/O.

1.3 Полезные ссылки

- [Официальная документация: asyncio](#)
- [Asyncio в Python: Полное руководство](#)

1.4 Задача на проверку

"Асинхронный парсер"

Напишите асинхронную функцию `fetch_data(api_id: int)`, которая имитирует сетевой запрос с помощью `asyncio.sleep(1)` и возвращает строку `f"Данные из API {api_id}"`. Напишите главную функцию `main()`, которая запускает конкурентно 3 таких запроса (с ID от 1 до 3) с помощью `asyncio.gather()`, замеряет общее

время выполнения (оно должно быть около 1 секунды, а не 3) и выводит список результатов.

Модуль 2: Поток (Threading) и Конкурентность

Что нужно уметь: Работать со стандартной библиотекой `threading`, использовать пул потоков, избегать Race Conditions с помощью примитивов синхронизации и помнить про ограничения GIL.

2.1 Чек-лист знаний

- [] Создание и запуск потоков через `threading.Thread(target=...)`.
- [] Ожидание завершения потока с помощью метода `.join()`.
- [] Понятие глобальной блокировки интерпретатора (GIL – Global Interpreter Lock) и его влияние на потоки в Python.
- [] Использование пула потоков `ThreadPoolExecutor` из модуля `concurrent.futures`.
- [] Защита общих данных от состояния гонки (Race Condition) с помощью `threading.Lock()`.

2.2 Глоссарий

- **Поток (Thread)** – наименьшая единица выполнения внутри одного процесса, разделяющая общую память с другими потоками этого процесса.
- **Состояние гонки (Race Condition)** – ошибка проектирования многопоточной системы, при которой несколько потоков одновременно модифицируют общие данные, приводя к непредсказуемому результату.

2.3 Полезные ссылки

- [Официальная документация: threading](#)
- [Пул потоков: ThreadPoolExecutor](#)

2.4 Задача на проверку

"Многопоточный счетчик"

Создайте глобальную переменную `counter = 0` и объект `lock = threading.Lock()`. Напишите функцию `increment()`, которая внутри

цикла (100 000 повторений) безопасно увеличивает `counter` на 1, используя контекстный менеджер `with lock:`. Запустите эту функцию в двух параллельных потоках, дождитесь завершения обоих через `.join()` и убедитесь, что итоговое значение `counter` равно ровно 200 000.

Модуль 3: Многопроцессность (Multiprocessing) и Параллелизм

Что нужно уметь: Обходить GIL для выполнения CPU-bound задач, распределять вычисления по ядрам процессора и понимать разницу в изоляции памяти.

3.1 Чек-лист знаний

- [] Создание процессов через `multiprocessing.Process` и пул процессов через `ProcessPoolExecutor`.
- [] Понимание природы CPU-bound задач (тяжелая математика, обработка графики, криптография).
- [] Полная изоляция памяти процессов (почему глобальные переменные не разделяются между процессами без спец. инструментов).
- [] Межпроцессное взаимодействие (IPC) с помощью `Queue` и `Pipe`.

3.2 Справочная таблица различий

Подход	Подходит для	Память	Ограничен GIL?	Накладные расходы
Asyncio	Высокий I/O (Web-скрейпинг, чаты)	Общая (один поток)	Нет (GIL не мешает)	Очень низкие
Threading	Умеренный I/O (Работа с файлами)	Общая для процесса	Да (нет истинного параллелизма кода)	Низкие

Multiprocessing	Вычисления (CPU-bound)	Изолированная	Нет (каждый процесс – свой GIL)	Высокие (запуск ОС-процесса)
------------------------	------------------------	---------------	---------------------------------	------------------------------

3.3 Полезные ссылки

- [Официальная документация: multiprocessing](#)

3.4 Задача на проверку

"Параллельный вычислитель"

Напишите функцию `compute_square(n: int) -> int`, возвращающую квадрат числа. В функции `main()` инициализируйте `ProcessPoolExecutor`. С помощью метода `.map()` параллельно вычислите квадраты для списка чисел `numbers = [1, 2, 3, 4, 5]`. Выведите полученный результат.

СБОРНИК ПРОВЕРОЧНЫХ ТЕСТОВ

Тест 1: Синхронность vs Асинхронность

Часть А: Выберите один правильный ответ

1. Что произойдет, если вызвать асинхронную функцию `async def my_func(): return 42` обычным способом: `result = my_func()`?
 - а) В переменную `result` запишется число 42.
 - б) Программа сразу упадет с ошибкой `TypeError`.
 - в) В переменную `result` запишется объект корутины, а код функции не выполнится.
 - г) Функция выполнится синхронно.
2. Какое ключевое слово используется для передачи управления обратно в Event Loop и ожидания результата другой корутины?
 - а) `yield`
 - б) `await`
 - в) `defer`
 - г) `pause`
3. Какая функция модуля `asyncio` предназначена для конкурентного выполнения списка корутин и ожидания их всех?
 - а) `asyncio.run()`
 - б) `asyncio.wait_all()`
 - в) `asyncio.gather()`
 - г) `asyncio.thread()`

4. Для каких задач асинхронное программирование дает наибольший выигрыш в производительности?

- а) Вычисление чисел Фибоначчи или майнинг криптовалюты.
- б) Перемножение огромных матриц.
- в) Ожидание ответов от сторонних веб-серверов или баз данных (I/O-bound).
- г) Рендеринг 3D-видео.

5. Что делает вызов `asyncio.run(main())`?

- а) Компилирует код функции `main` в машинный код.
- б) Создает новый цикл событий (Event Loop), запускает корутину `main()` и закрывает цикл после завершения.
- в) Запускает корутину `main` в отдельном системном процессе.
- г) Проверяет код на наличие синтаксических ошибок.

Часть Б: Задания со свободным ответом (напишите код)

6. Напишите объявление асинхронной функции с именем `fetch_page`, которая принимает аргумент `url: str`.

7. Как симитировать неблокирующую паузу в 2.5 секунды внутри корутины? Напишите строку кода.

8. Исправьте ошибку в коде:

```
async def main():  
    asyncio.sleep(1)  
    print("Done")
```

9. Какого типа объект возвращает выражение `async def f(): pass` при вызове `type(f())`?

10. Каким методом можно обернуть синхронную блокирующую функцию, чтобы запустить её внутри Event Loop без блокировки всего цикла (подсказка: `loop.run_in_...`)?

Тест 2: Поток (Threading)

Часть А: Выберите один правильный ответ

1. Что такое GIL (Global Interpreter Lock) в CPython?
 - а) Механизм автоматического форматирования кода.
 - б) Блокировка, гарантирующая, что только один поток выполняет байт-код Python в один момент времени.
 - в) Способ защиты базы данных от несанкционированного доступа.
 - г) Модуль для шифрования строк.
2. Какой метод класса `threading.Thread` нужно вызвать, чтобы запустить выполнение созданного потока?
 - а) `run()`
 - б) `execute()`
 - в) `start()`
 - г) `join()`
3. Что делает метод `thread.join()`?
 - а) Немедленно убивает поток `thread`.
 - б) Объединяет память двух разных потоков.
 - в) Блокирует вызывающий поток (обычно главный) до тех пор, пока поток `thread` не завершит свою работу.
 - г) Перезапускает поток сначала.
4. Какая проблема может возникнуть, если два потока одновременно пытаются изменить одну и ту же глобальную переменную без синхронизации?
 - а) Ошибка компиляции.

б) Состояние гонки (Race Condition).

в) Deadlock (Взаимная блокировка).

г) Сегментация памяти.

5. Каков тип примитива синхронизации, который работает по принципу контекстного менеджера и может быть "захвачен" (**acquire**) и "отпущен" (**release**)?

а) `threading.Semaphore`

б) `threading.Event`

в) `threading.Lock`

г) `threading.Condition`

Часть Б: Задания со свободным ответом (напишите код)

6. Напишите код создания потока, который должен запустить функцию **worker** без аргументов.

7. Как передать кортеж аргументов **(10, 20)** в поток при его инициализации через класс **Thread**? Напишите фрагмент кода.

8. Исправьте ошибку (поток создали, но забыли кое-что важное):

```
import threading
```

```
t = threading.Thread(target=print, args=("Hello",))
```

```
# Код должен напечатать Hello, но программа завершается без вывода
```

9. С помощью какого менеджера контекста (**with**) рекомендуется захватывать объект блокировки **lock = threading.Lock()**?

Напишите пример структуры.

10. Поток в Python эффективны для CPU-bound или для I/O-bound задач?

Тест 3: Многопроцессность (Multiprocessing)

Часть А: Выберите один правильный ответ

1. В чем главное отличие процессов (**multiprocessing**) от потоков (**threading**) в Python?
 - а) Процессы работают быстрее с сетевыми запросами.
 - б) Процессы делят между собой общую оперативную память напрямую.
 - в) Каждый процесс имеет собственный независимый экземпляр интерпретатора Python и свою изолированную память, обходя GIL.
 - г) Процессы нельзя запускать на Unix-системах.
2. Какой пул из модуля **concurrent.futures** следует использовать для эффективного параллельного вычисления математических задач на многоядерном процессоре?
 - а) ThreadPoolExecutor
 - б) ProcessPoolExecutor
 - в) AsyncIOExecutor
 - г) LoopPoolExecutor
3. Что произойдет, если изменить обычную глобальную переменную внутри дочернего процесса **multiprocessing.Process**?
 - а) Она изменится и в главном процессе, так как память общая.
 - б) Программа вылетит с ошибкой **MemoryError**.
 - в) Изменение произойдет только внутри изолированной памяти дочернего процесса; в главном процессе переменная останется прежней.

г) Изменение заблокирует все остальные процессы.

4. Какой объект из модуля `multiprocessing` используется для безопасной передачи данных между процессами по принципу FIFO (First In, First Out)?

а) `multiprocessing.Pipe`

б) `multiprocessing.Queue`

в) `multiprocessing.Lock`

г) `multiprocessing.Pool`

5. Какое обязательное условие необходимо соблюдать на ОС Windows при работе с модулем `multiprocessing` во избежание бесконечной рекурсивной генерации подпроцессов?

а) Код создания процессов должен находиться под защитой `if __name__ == '__main__':`.

б) Нельзя использовать лямбда-функции.

в) Все функции должны быть асинхронными.

г) Нужно отключать антивирус.

Часть Б: Задания со свободным ответом (напишите код)

6. Напишите импорт класса пула процессов из модуля `concurrent.futures`.

7. Защитите конструкцию вызова процесса классической проверкой точки входа скрипта.

8. Каким методом у объекта `multiprocessing.Queue` можно извлечь элемент из очереди?

9. Перечислите типы задач (одним термином), для которых строго показано использование `multiprocessing` вместо `asyncio`.

10. Напишите код запуска процесса `p = Process(target=task)`, за которым следует ожидание его завершения.

ОТВЕТЫ И РАЗБОР ЗАДАНИЙ

Тест 1: Синхронность vs Асинхронность (asyncio)

- 1: в) Вызов `async def` возвращает корутину. Чтобы она выполнялась, её нужно передать в Event Loop или вызвать с `await`.
- 2: б) `await` временно приостанавливает выполнение текущей корутины и отдает управление обратно в цикл событий.
- 3: в) `asyncio.gather` принимает несколько awaitable-объектов и выполняет их конкурентно.
- 4: в) I/O-bound задачи проводят большую часть времени в ожидании внешних ответов. Асинхронность позволяет не простаивать потоку.
- 5: б) Это стандартная точка входа для любого асинхронного приложения на Python.
- 6: `async def fetch_page(url: str):`
- 7: `await asyncio.sleep(2.5)`
- 8: Забыли ключевое слово `await` перед вызовом неблокирующего сна: `await asyncio.sleep(1)`.
- 9: `<class 'coroutine'>`
- 10: `loop.run_in_executor()`

Тест 2: Потоки (Threading)

1: б) GIL не позволяет нескольким потокам одного процесса параллельно выполнять нативный код Python на разных ядрах CPU.

2: в) Метод `.start()` инициализирует поток в ОС и запускает переданный таргет. Вызов `.run()` просто выполнит код синхронно в текущем потоке.

3: в) `.join()` заставляет программу подождать, пока поток полностью сделает свою работу, прежде чем идти дальше.

4: б) Состояние гонки (Race Condition). Операции над переменными не атомарны, потоки могут перезаписать шаги друг друга.

5: в) `threading.Lock` обеспечивает взаимное исключение (mutual exclusion).

6: `t = threading.Thread(target=worker)`

7: `t = threading.Thread(target=func, args=(10, 20))`

8: Поток был создан, но не запущен. Требуется добавить строчку `t.start()`.

9: `with lock:` (Это автоматически вызывает `acquire()` на входе и `release()` на выходе из блока).

10: Для I/O-bound задач (так как во время операций ввода-вывода GIL отпускается).

Тест 3: Многопроцессность (Multiprocessing)

1: в) Так как у каждого процесса свой интерпретатор, они могут параллельно вычисляться на разных ядрах процессора, полностью обходя GIL.

2: б) `ProcessPoolExecutor` идеально утилизирует многоядерность для вычислительных задач.

3: в) Память процессов полностью изолирована. Для обмена данными нужны спец. структуры (`Queue`, `Value`).

4: б) Очередь (`Queue`) потоко- и процессоробезопасна, реализует интерфейс FIFO.

5: а) Без `if __name__ == '__main__':` Windows начнет бесконечно импортировать этот же файл для новых процессов, что приведет к краху системы.

6: `from concurrent.futures import ProcessPoolExecutor`

7: `if __name__ == '__main__':`

8: `queue.get()`

9: CPU-bound.

10: `p.start(); p.join()`